LLM 学习笔记

夏同

2026 年 3 月 1 日

目录

第一章 Text to SQL 生成自我修正指南 **3**

1.1 相关工作 4

1.1.1 用于 Text-to-SQL 的 LLM 4

1.1.2 Text-to-SQL 中的自我修正 4

1.2 MAGIC 方法 4

1.2.1 任务定义 4

1.2.2 三个智能体 4

1.2.3 反馈-修正循环 5

1.2.4 修订智能体指令 5

1.3 实验设置 5

1.3.1 数据集 5

1.3.2 评估指标 5

1.3.3 基线方法 5

1.4 结果 6

1.4.1 RQ1：与基线方法的有效性对比 6

1.4.2 RQ2：反馈数量的影响 6

1.4.3 RQ3：管理者智能体的作用 7

1.5 讨论 8

1.5.1 对其他方法的适用性 8

1.5.2 指南中示例的来源 8

1.5.3 数据库分析 8

1.6 结论 8

第二章 揭示思维链推理对直接偏好优化的影响：来自 Text-to-SQL 的教训 **9**

2.0.1 背景知识：偏好优化 9

2.0.2 从执行反馈中学习 10

2.1 引言 10

2.1.1 碰壁 11

2.1.2 反思 11

2.1.3 验证猜想：给数据补上 CoT 11
2.1.4 刨根问底：CoT 到底起了什么作用？ 12
2.2 方法论 12
2.2.1 思维链合成 12
2.2.2 直接偏好优化（DPO） 13
2.3 实验设置 14
2.3.1 数据集 14
2.3.2 评估指标 14
2.3.3 推理策略 14
2.3.4 实现细节 15
2.4 实验结果 15
2.4.1 主要结果：Bird 开发集 15
2.4.2 错误分析：CoT 如何帮助 DPO 纠正错误？ 16
2.4.3 Spider 及其变体的结果 16
2.4.4 更好的判别能力 17
2.4.5 更好的训练稳定性 17
2.4.6 更好的可扩展性 17
2.5 前景：集成到现有 Text-to-SQL 框架中 18
2.6 结论 18
2.7 局限性 18

第一章　Text to SQL 生成自我修正指南

文本到 SQL 中的自我修正是提示大型语言模型（LLM）修改其先前错误生成的 SQL 查询的过程。当前的小样本方法通过人工分析 LLM 在训练数据上的常见错误来设计自我修正指南——这是一项耗时任务，且受限于人类识别所有错误模式的能力。

这里提出 MAGIC，一种新颖的多智能体方法，可自动化自我修正指南的创建。MAGIC 采用三个专门智能体（管理者、修正者和反馈者），它们基于训练数据中 LLM 方法的失败案例协作，迭代生成并精炼针对 LLM 错误的自我修正指南并在推理时集成到初始方法中，帮助其避免常见错误。这里的实验表明，MAGIC 生成的指南优于人类专家编写的指南，并增强了修正的可解释性。

贡献包括：（i）建立文本到 SQL 的自我修正指南生成任务，提出 MAGIC 方法，其生成的指南优于人类编写；（ii）系统分析不同场景下自我修正的效果；（iii）使用相同 GPT-4 版本复现所有基线，提供全面可比较的见解；（iv）开源所有代码。

基础概念介绍

- **Text-to-SQL**：将日常语言描述的问题（例如“去年销量最高的产品是什么？”）自动转换成能在数据库上执行的 SQL 查询语句的任务。
- **大型语言模型（LLM）**：像 GPT-4 这样的大规模神经网络，能够理解和生成自然语言及代码。
- **自我修正**：让模型自己检查先前生成的 SQL 是否正确，如果不正确就尝试修改，类似于人类写完答案后再检查一遍。
- **智能体（Agent）**：这里指专门负责某项工作的 AI 程序模块。MAGIC 中有三个智能体，各自分工合作。
- **自我修正指南**：一组提示或规则，告诉模型如何修正常见的错误，例如“当问题要求‘最多的’时，记得用 ORDER BY 和 LIMIT 1”。
- **数据集**：**Spider** 和 **BIRD** 是两个广泛使用的评测数据集，包含许多自然语言问题及其对应的正确 SQL，用来测试 text-to-SQL 系统的准确率。
- **执行准确率（EX）**：将模型生成的 SQL 在数据库上运行，看结果是否和标准答案的运行结果一致。一致就算正确。
- **有效效率分数（VES）**：在正确的基础上，还考虑 SQL 的执行速度，速度越快分数越高

（只在 BIRD 这种大数据库上有意义）。

- **基线方法**：用来和新方法对比的已有方法，比如 Self-Debugging、Self-Consistency 等。

1.1 相关工作

1.1.1 用于 Text-to-SQL 的 LLM

早期方法利用 LLM 的零样本上下文学习能力生成 SQL。后续模型通过任务分解和思维链等技术提升性能，如 DAIL-SQL、MAC-SQL、C3、自我一致性和从少到多提示。此外，还有研究专注于微调 LLM。这里选择 DIN-SQL 作为初始系统，因其高效、开源且包含人类编写的自我修正指南，适合与 MAGIC 自动生成的指南对比。

1.1.2 Text-to-SQL 中的自我修正

自我修正研究在多个领域已有综述，但在 Text-to-SQL 中相对有限。

- Self-debugging 生成解释用于修正；
- DIN-SQL 设计人类编写指南并修正所有初始 SQL；
- 自我一致性通过生成多个候选并投票；
- MAC-SQL 的精炼器使用执行错误作为修正信号。

这里是首个通过生成自我修正指南来解决 Text-to-SQL 自我修正的研究，也是首个在 Text-to-SQL 或其他领域使用多智能体方法设计自我修正指南的工作。。基于 LLM 的智能体研究广泛，如 AutoGPT、OpenAgents、AutoGen。但在 Text-to-SQL 中应用较少，MAC-SQL 是唯一的多智能体协作框架。受此启发，MAGIC 采用多智能体协作框架，自动分析 SQL 预测失败并生成指南。

1.2 MAGIC 方法

1.2.1 任务定义

给定一组自然语言问题及其对应的数据库模式和初始生成的错误 SQL 查询，任务是生成一个针对 LLM 错误的自我修正指南。指南生成所用的数据不应与评估数据重叠，以防过拟合。

1.2.2 三个智能体

MAGIC 包含三个智能体，它们通过失败案例迭代交互：

- **管理者智能体**：协调整个过程，存储成功的反馈，并聚合反馈以生成指南。
- **反馈智能体**：通过与标准答案对比，解释错误 SQL 中的错误。
- **修正智能体**：根据提供的反馈修改错误 SQL。

1.2.3 反馈-修正循环

对于每个问题、标准答案 SQL s^{gt} 和错误 SQL s'：

1. 管理者请求反馈智能体通过与 s^{gt} 对比解释 s' 中的错误。
2. 管理者整合反馈，并请求修正智能体根据反馈修改 s'。
3. 修正智能体生成修改后的 SQL。
4. 管理者执行修改后的 SQL 并将结果与 s^{gt} 对比。
5. 当修改成功或达到最大迭代次数（5 次）时循环结束。

1.2.4 修订智能体指令

在第一次迭代中，管理者使用预定义提示与反馈和修正智能体交互。如果修正失败，管理者会根据先前的响应、问题、s' 和 s^{gt} 修订这些预定义提示。这里发现此步骤对管理者适应智能体交互至关重要。

对于每次成功的修改，管理者存储对应的反馈。当累积到 10 个成功反馈的批次时，管理者使用预定义提示生成或更新自我修正指南。过程在 2 小时内完成。先前方法需要大量人工工作，而 MAGIC 可在 2 小时内高效生成经验证更好的指南。

管理者智能体在生成指南时可访问所有信息，但其他智能体访问受限。修正智能体始终只接收 s'，确保成功修正表明反馈能有效覆盖错误。这保证了生成的指南能防止初始系统的错误。

1.3 实验设置

1.3.1 数据集

- **Spider**：包含 10,181 个问题，5,693 个独特 SQL 查询，涵盖 200 个数据库，分为训练、开发和测试集。
- **BIRD**：包含 12,751 个问题-SQL 对，涵盖 95 个大数据库（33.4 GB）和 37 个专业领域，引入外部知识增加复杂度。

1.3.2 评估指标

- **执行准确率（EX）**：比较预测 SQL 的执行结果与标准答案 SQL 的执行结果。
- **有效效率分数（VES）**：评估正确查询的执行效率。对于 Spider，由于数据库小，执行时间差异可忽略，故不报告 VES。

1.3.3 基线方法

无指南的自我修正方法：

- Self-Debugging：生成解释用于自我修正。
- Self-Consistency：生成 20 个 SQL 查询，通过投票选择最终 SQL。
- Multiple-Prompt：重新排列表格候选，生成 20 种组合，采用投票机制。

带指南的自我修正方法：

- Human Expert G (MAC-SQL)：针对执行错误的人类编写指南。
- Human Expert G (DIN-SQL)：DIN-SQL 的人类编写指南。

所有基线均使用相同 GPT-4 版本复现。

1.4 结果

1.4.1 RQ1：与基线方法的有效性对比

表 1.1: BIRD 数据集执行准确率（%）——数值越高越好

方法	自我修正应用场景		
	修正错误 SQL (已知 SQL 错了)	仅当 SQL 执行 (出错时修正)	不管对错 (都尝试修正)
无自我修正	56.52	56.62	56.52
无指南的自我修正			
Self-Debugging	57.24	56.78	56.58
Self-Consistency	58.08	56.58	57.17
Multiple-Prompt	57.86	56.65	57.30
带指南的自我修正			
Human Expert G (MAC-SQL)	56.60	56.58	46.14
Human Expert G (DIN-SQL)	57.76	56.98	55.80
MAGIC G (Ours)	**59.13**	**57.32**	**58.55**

MAGIC 生成的指南在两个数据集上均优于所有基线。注意：在“修正所有 SQL”场景中，MAC-SQL 的人类指南反而降低了准确率（从 56.52 降到 46.14），因为盲目修正可能把对的改错。这说明指南质量至关重要。另外，初始系统中只有 46 个 SQL 执行出错，所以“修正执行错误的 SQL”场景下提升空间有限。

1.4.2 RQ2：反馈数量的影响

聚合 10 个批次（100 条反馈）后，MAGIC 生成的指南已优于人类编写的指南，继续增加反馈不再提升效果。这表明 100 条反馈足以覆盖常见错误模式。

表 1.2: Spider 数据集执行准确率（%）——数值越高越好

方法	EX
无自我修正	78.62
Self-Consistency	81.64
Multiple-Prompt	80.22
Human's G (MAC-SQL)	81.15
Human's G (DIN-SQL)	80.35
MAGIC's G (Ours)	**85.66**

表 1.3: 使用不同数量的反馈批次生成指南的效果（BIRD）

累积反馈批次数	0	1	5	10	39（全部）
执行准确率 EX（%）	56.52	57.4	58.8	59.13	59.13

1.4.3 RQ3：管理者智能体的作用

管理者智能体负责协调、存储反馈并修订指令。实验对比了有无管理者的效果。无管理者时，只有反馈和修正智能体直接循环（类似批评-精炼），不生成指南。

表 1.4: 管理者智能体对修正效果的影响（训练集）

设置	BIRD			Spider	
	EX↑	VES↑	平均迭代次数 ↓	EX↑	平均迭代次数 ↓
最大迭代次数 =2					
无管理者	73.01	68.29	1.17	81.40	1.11
有管理者	78.94	80.22	1.12	86.52	1.05
最大迭代次数 =5					
无管理者	78.60	79.58	2.87	83.28	2.22
有管理者	81.81	84.19	2.41	91.78	2.15

EX：执行准确率（越高越好）；VES：效率分数（越高越好）；平均迭代次数：修正成功所需的平均循环次数（越少越好）。

有管理者时，不仅修正准确率更高，生成的 SQL 也更高效，且所需迭代次数更少。这是因为管理者会在修订提示时加入优化指令。

1.5 讨论

1.5.1 对其他方法的适用性

这里将 MAGIC 生成的指南应用到其他 text-to-SQL 方法上，看是否也能提升它们的表现。

表 1.5: MAGIC 指南在不同方法上的表现（BIRD 开发集）

方法	原准确率 EX（%）	加 MAGIC 指南后 EX（%）
Zero-shot GPT-4	40.18	48.19
Few-shot CoT GPT-4	45.66	50.38
DIN-SQL	56.52	59.13
MAC-SQL	59.39	61.92

MAGIC 的指南能普遍提升各种方法的准确率，说明其具有一定的通用性。

1.5.2 指南中示例的来源

这里检查了生成指南中的示例是否直接抄自反馈，发现并非如此。管理者智能体会把多条反馈综合起来，创造出新的示例，避免过拟合。

1.5.3 数据库分析

不同数据库上 MAGIC 指南的提升幅度不同。有些数据库（如涉及复杂查询的）提升更大，有些则较小。这可能与 LLM 对特定领域知识的掌握程度有关，提示未来可以针对数据库特点定制指南。

1.6 结论

MAGIC 引入了一种新颖的多智能体方法，用于自动生成 Text-to-SQL 中的自我修正指南。它克服了手动创建指南的局限性，优于人类编写的指南，并改进了强大的小样本方法。这项工作展示了利用 LLM 生成自身自我修正指南的潜力，并将指南生成确立为一个值得专注研究的重要任务。所有代码已开源。

第二章　揭示思维链推理对直接偏好优化的影响：来自 Text-to-SQL 的教训

2.0.1　背景知识：偏好优化

偏好优化是一种让模型与人类偏好对齐的技术，它通常需要一个包含“好/坏”回答对比的训练集。最经典的方法是**基于人类反馈的强化学习（Reinforcement Learning from Human Feedback, RLHF）**，它先训练一个奖励模型来模拟人类偏好，再用强化学习（如 PPO）优化语言模型。然而，RLHF 流程复杂且不稳定。

直接偏好优化（Direct Preference Optimization, DPO）用一个巧妙的重参数化技巧，把奖励模型直接融进了损失函数，从而绕过了强化学习步骤，让偏好优化能像监督学习一样简单稳定。DPO 的目标是最大化正确回答与错误回答之间的对数似然差异，同时确保模型参数不会偏离原始模型太远。核心损失函数如下：

$$\mathcal{L}_{\mathrm{DPO}}(\pi_\theta;\pi_{\mathrm{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w|x)}{\pi_{\mathrm{ref}}(y_w|x)} - \beta\log\frac{\pi_\theta(y_l|x)}{\pi_{\mathrm{ref}}(y_l|x)}\right)\right]$$

其中 π_θ 是训练中的模型，π_{ref} 是参考模型（通常是 SFT 后的模型），y_w 和 y_l 分别代表正确（偏好）和错误（被拒绝）的回答，β 是个超参数，控制对偏离参考模型的惩罚强度，σ 是 sigmoid 函数。

这个公式是 DPO（直接偏好优化）的核心损失函数，它的设计目的是让模型学会更倾向于生成“好”的回答、远离“坏”的回答，同时保证模型不会偏离原本的参考模型太远。下面我们逐部分拆解它的含义。本质上是在做对比学习，拉近模型与好回答的距离，推远与坏回答的距离。但与普通对比学习不同，这里的“距离”是用相对于参考模型的概率比来度量的。公式中的 $\beta\log\frac{\pi_\theta(y|x)}{\pi_{\mathrm{ref}}(y|x)}$ 可以看作模型为输出 y 分配的隐式奖励。损失函数就是在最大化好回答与坏回答之间的奖励差。由于参考模型 π_{ref} 固定，优化过程中模型如果为了增大差值而过度调整参数，会导致概率比变得极大，但 β 的存在会抑制这种偏离（因为差值中包含了 π_{ref} 的对数概率，模型无法随意增加好回答的概率而不影响其他输出）。

传统 RLHF 需要先训练一个奖励模型，再用 PPO 等强化学习算法优化语言模型，流程复杂且不稳定。DPO 直接通过这个损失函数，将偏好优化转化为监督学习问题，训练过程简单、高效，且效果往往不逊于 RLHF。这个公式通过比较当前模型与参考模型在好坏回答上的概率差异，引导模型在保持基本能力的同时，学会区分优劣输出，从而实现与人类偏好对齐。它是 DPO 算法的核心，也是实现偏好优化“简单化”的关键。

此后，研究者们又提出了许多 DPO 的变体，比如 SimPO、KTO、IPO 等，它们在不同场景下各有千秋。在本研究中，研究人员采用最经典的 DPO 作为代表算法，因为它效果已经过广泛验证，而且实现简单。值得一提的是，目前只有一项研究（SENSE, Yang et al., 2024c）尝试把 DPO 用到 Text-to-SQL 上。不过他们的方法跟研究人员有本质区别：SENSE 是用小模型（1B）为大模型生成偏好数据，而研究人员遵循标准 DPO 流程，直接从 SFT 模型采样构建偏好数据。更重要的是，SENSE 仍然用 SQL 查询作为训练标签，而研究人员的研究则探索了将 CoT 解决方案作为训练标签的关键作用。

2.0.2　从执行反馈中学习

在 LLM 火起来之前，通过执行反馈进行学习已经在代码生成任务中崭露头角。比如用单元测试结果作为奖励信号，通过强化学习优化代码生成模型。类似的工作还有 RLTF (Liu et al., 2023)、StepCoder (Dou et al., 2024) 等。在 Text-to-SQL 领域，执行反馈天然可用——因为 SQL 查询可以在数据库上执行并与标准结果对比。然而，现有工作要么没有充分利用执行反馈，要么在在线学习方式中面临巨大的计算开销（执行 SQL 需要大量内存和 CPU 资源）。本文是首个通过离线构建偏好数据（利用执行反馈标记正确/错误）并结合 DPO 实现稳定性能提升的工作。

2.1　引言

近年来，Text-to-SQL 在自然语言处理和数据库研究领域掀起了一股热潮。随着大型语言模型（LLM）的横空出世，解决 Text-to-SQL 的路径主要分成了两条。

第一条是**基于提示的方法**：直接向 GPT-4、Gemini 这样的闭源巨头提问，让它们生成 SQL。理论可行，但实际操作中往往需要设计复杂的提示，甚至动用多个智能体协作，成本高、响应慢，而且把数据送到外部服务还可能引发隐私担忧。

第二条是**监督微调（SFT）方法**：在公开的 Text-to-SQL 数据集（比如 Spider、Bird）上对开源模型进行微调，让它们学会把问题映射到 SQL。微调后的模型可以本地部署，成本低、速度快，数据也安全。可惜的是，SFT 的性能被高质量训练数据的稀缺性卡住了脖子——创建这些数据既烧钱又耗时，模型只能模仿训练集里的“标准答案”，一旦遇到没见过的复杂问题就容易露馅了。

为突破监督微调的瓶颈，也许可以借助一种叫“偏好优化”的算法来增强 SFT 模型的表现，如**直接偏好优化（Direct Preference Optimization, DPO）**。和 SFT 只让模型学正确答案不同，DPO 会喂给模型一些“偏好对”——比如一个正确回答和一个错误回答，然后训练模型更倾向于生成正确的，同时远离错误的。这招在数学解题和代码生成这类需要复杂推理的任务中屡试不爽。但在 Text-to-SQL 领域却几乎没人用，这是为什么呢？

2.1.1　碰壁

为了找到答案，研究人员在一个颇具挑战性的 Text-to-SQL 基准——Bird 数据集上进行试验。Bird 的每个样本都包含一个问题、一个数据库和对应的标准 SQL 查询。挑 10 个参数规模从 6.7B 到 15B 的开源 LLM，按标准 DPO 流程一步步来：先做监督微调，让模型学会从问题和数据库信息生成 SQL；然后用微调好的模型为每个训练样本采样多个 SQL 查询，再到数据库上执行，把结果正确的标记为“正确”，错误的标为“错误”，配对成偏好对；最后用 DPO 损失在这些偏好对上进行一通优化。

实验结果却不尽如人意。DPO 压根没能稳定提升性能——10 个模型里，有 6 个的准确率反而下滑。研究人员试遍了各种挽救的招数：调超参数、往 DPO 损失里加 SFT 损失、把 DPO 换成其他偏好优化算法（比如 KTO）、甚至学别人的方法用小模型生成偏好数据。可折腾一圈下来，最好的情况也不过是不到 1.5% 的微小提升。

2.1.2　反思

撞了南墙之后，研究人员开始反思：为什么 DPO 在数学和代码任务中屡试不爽，到了 Text-to-SQL 就水土不服了呢？对比两类任务的数据集，研究人员发现了它们之间的一个关键差异。数学和代码数据集，比如 MATH、GSM8K、CodeUltraFeedback，通常不只给最终答案，还会附上详细的解题步骤或推理过程，也就是所谓的“思维链”（Chain-of-Thought, CoT）解决方案。思维链推理之所以能释放 DPO 的潜力，是因为它能缓解“奖励破解”（模型为了得高分而走捷径），增强模型区分好坏的能力，并让训练过程更好地规模化扩展。这些推理过程就像一座桥梁，把问题和最终答案连接起来，让模型在 SFT 和 DPO 训练中能更好地理解背后的逻辑，从而举一反三。

反观 Text-to-SQL 数据集，像 Bird、Spider、WikiSQL，它们都只给最终答案——标准的 SQL 查询语句，中间怎么一步步推出来的完全没有。模型在 SFT 阶段只能死记硬背 SQL 模板，在 DPO 阶段也只能在最终 SQL 层面比对比对，根本缺乏对推理过程的监督。这就像只给学生答案而不教他们解题思路。那么如果为 Text-to-SQL 数据自动补上思维链，应该就会实现 DPO 在 Text-to-SQL 任务上稳定且显著的性能提升。

2.1.3　验证猜想：给数据补上 CoT

为了验证这个想法，研究人员设计了一个流程来研究 CoT 究竟如何影响 DPO 在 Text-to-SQL 中的表现。研究人员找来一个 LLM 充当“CoT 合成器”，让它为现有的 Text-to-SQL 数据集自动生成逐步的 CoT 解决方案。这个合成器会接收数据库信息、问题以及标准 SQL 查询作为输入，然后输出详细的推理步骤，一步步展示如何将问题转化为 SQL。用这种方法，研究人员以极低的人力成本构建了 CoT 增强的 Bird 数据集。

接着，研究人员在完全相同的实验设置下，对 CoT 增强的数据集进行了 SFT 和 DPO 训练。结果如图 1 所示：**加上 CoT 之后，所有 10 个评估模型在 DPO 阶段都获得了显著且稳**

定的性能提升！研究人员还不放心，又把评估范围扩展到 Spider、SpiderDK、Spider-Syn 等多个 Text-to-SQL 基准，结果一致，无一例外。

2.1.4　刨根问底：CoT 到底起了什么作用？

为了搞清楚 CoT 为什么这么重要，研究人员深入分析，总结出三个核心发现。第一，引入 CoT 显著减少了 DPO 训练中常见的“奖励破解”现象——也就是模型通过生成看似合理但实际错误的输出来骗取高分，让训练变得更加稳定有效。第二，CoT 增强了 DPO 作为隐式奖励模型的判别能力，让它能更准确地判断一个回答是好是坏。第三，CoT 提高了 DPO 的可扩展性，无论增加偏好数据量，还是在推理时加大采样预算，CoT 增强的 DPO 都能持续受益。

2.2　方法论

本节将详细介绍研究人员提出的流程，它包含三个环环相扣的步骤：思维链（CoT）合成、监督微调（SFT）、直接偏好优化（DPO）。图 2 展示了整个流程的概览。

2.2.1　思维链合成

现有的 Text-to-SQL 数据集，比如 Bird 和 Spider，都只提供最终答案——标准 SQL 查询，而没有从问题到 SQL 的中间推理过程。为了让模型学会像人一样一步步思考，研究人员首先需要为每个训练样本生成 CoT 解决方案。

为什么非得要 CoT 呢？因为对于一个复杂的 Text-to-SQL 问题，人类解决它通常需要经历一系列步骤：先理解问题到底在问什么，再识别出涉及哪些数据库表和列，然后考虑有什么过滤条件，需不需要聚合，最后才能构建出完整的 SQL 语句。如果模型也能学习这样的中间步骤，它就能更好地理解问题的逻辑，而不是简单地死记硬背 SQL 模板。更重要的是，在 DPO 训练中，有了 CoT，模型就可以在词元级别获得更细粒度的反馈，而不是只在最终 SQL 层面判断对错，这能有效缓解奖励破解的问题。

那怎么生成 CoT？为了尽量减少人工标注的成本，研究人员请来一个 LLM 当“CoT 合成器”。具体来说，对于训练集中的每个样本（包括问题 q，数据库信息 d，标准 SQL s），研究人员用 GPT-4o-mini（OpenAI 2024b）生成 K 个多样化的 CoT 解决方案。每个 CoT 解决方案 c 都是一段文本，它会一步步展示如何从问题出发，结合数据库模式，最终推导出正确的 SQL 查询。因为最终答案（标准 SQL）已经提供给了合成器，所以生成的 CoT 解决方案既可靠又准确，完全可以放心地用于后续的 SFT 和 DPO 训练。需要注意的是，研究人员并不要求所有生成的 CoT 都完美无瑕，但通过引导模型生成多样化的推理路径，可以为后续训练提供丰富的语义信息。在本工作中，研究人员设定 $K = 16$，也就是为每个原始样本生成 16 个 CoT 解决方案。

2.2.2　直接偏好优化（DPO）

SFT 虽然能让模型学会生成 CoT，但它本质上还是在模仿训练数据，可能仍然无法区分好的推理路径和坏的推理路径。DPO 则通过偏好数据进一步优化模型，让它更倾向于生成正确的回答。

偏好数据集构建

研究人员需要构建一个偏好数据集 D_P，其中每个样本包含一个问题 q、数据库信息 d、一个正确的 CoT 解决方案 c^+ 和一个错误的 CoT 解决方案 c^-。构建过程是这样的：首先，对于训练集中的每个样本，研究人员用参考模型 π_{SFT} 生成 N 个不同的 CoT 解决方案（研究人员取 $N = 16$）。接着，从每个生成的 CoT 解决方案里，用正则表达式提取出预测的 SQL 查询。

然后，把预测 SQL 和标准 SQL 分别在对应的数据库上执行，比较它们的执行结果（比如返回的记录是不是完全一致）。如果结果相同，就把这个 CoT 解决方案标记为“正确”；否则就标记为“错误”。最后，对于每个训练样本，如果它同时有至少一个正确和一个错误的 CoT 解决方案，研究人员就随机挑选一个正确和一个错误的，组成一个偏好对 (c^+, c^-)，并把它加入偏好数据集 D_P。因为并不是每个样本都能同时产生正确和错误的 CoT（比如模型可能全对或全错），最终得到的偏好数据集大小通常小于 SFT 数据集（以 Bird 为例，大约有 1.5k 到 2.5k 对）。

DPO 学习目标

DPO 的目标是让正确回答相对于错误回答的“优势”最大化，同时确保模型不会跑偏太远，离参考模型太远。它隐含地定义了一个奖励模型。对于任意输出 y，它的隐式奖励可以这样算：

$$r_\theta(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$$

这个奖励可以理解为模型对当前输出相对于参考模型的偏好程度。因为语言模型是自回归的，研究人员还可以把这个奖励分解到每个词元上：

$$r_\theta(x, y) = \sum_{t=1}^{T} \beta \log \frac{\pi_\theta(y_t|x, y_{<t})}{\pi_{\text{ref}}(y_t|x, y_{<t})}$$

这种分解让研究人员能在词元级别分析模型对每个生成步骤的“信心”，对于理解模型行为（比如奖励破解）特别有帮助。举个例子，如果模型在某个词元上给出了异常高的奖励，那可能意味着它找到了一条捷径，值得警惕。在后续的分析中，研究人员会用这个隐式奖励来评估 DPO 模型的判别能力和训练稳定性。

2.3　实验设置

为了验证研究人员的猜想，并全面评估 CoT 增强 DPO 的效果，研究人员精心设计了一系列实验。

2.3.1　数据集

研究人员的实验主要在以下几个 Text-to-SQL 基准上进行。首先是 **Bird (Li et al., 2024b)**，这是一个颇具挑战性的跨领域数据集，包含 9,428 个训练样本和 1,534 个开发样本，涉及 37 个专业领域的 95 个大型数据库。Bird 的数据库通常拥有大量真实数据行，因此执行结果反馈更为可靠，也更贴近实际应用场景。其次是 **Spider (Yu et al., 2018)**，它是最常用的 Text-to-SQL 基准，包含 7,000 个训练样本和 1,034 个开发样本，涵盖 138 个领域的 200 个数据库。

Spider 的 SQL 查询相对简单，数据库行数较少，执行结果反馈可能存在假阳性问题。为了评估模型的泛化能力，研究人员还在 Spider 的几个变体上进行了测试，包括 Spider-DK、Spider-Syn 、Spider-Realistic 以及 Dr.Spider。这些数据集通过对 Spider 开发集进行各种扰动（如同义词替换、数据库结构调整等），模拟了真实世界中的各种挑战。

2.3.2　评估指标

研究人员采用**执行准确率（Execution Accuracy, EX）**作为主要评估指标。EX 衡量的是预测 SQL 查询与标准 SQL 查询在数据库上执行后，返回的结果是否完全一致。对于 Spider 开发集，研究人员还额外报告**测试套件准确率（Test Suite Accuracy, TS）**。TS 通过在多个测试套件数据库实例上执行来减少假阳性，提供更稳健的评估。

2.3.3　推理策略

给定一个训练好的 Text-to-SQL 模型，研究人员用三种不同的推理策略来评估它的性能。

第一种是**贪心（Greedy）**，也就是使用温度为 0 的解码，每次选择概率最高的词元，生成单个确定性的响应。这是最常用的快速评估方式。

第二种是 **Pass@1**，研究人员使用温度为 1.0 采样一个响应，重复 16 次，计算平均准确率。这反映了模型在单次采样下的期望性能。

第三种是 **Maj@K**，研究人员使用温度为 1.0 采样 K 个响应（通常 $K = 16$），然后基于每个响应提取的 SQL 查询在数据库上的执行结果进行多数投票，将得票最多的查询作为最终预测。这种方法通常能提升准确率，因为它聚合了多次采样的共识。

2.3.4 实现细节

在基础模型的选择上，研究人员挑了 10 个有代表性的开源 LLM，参数范围从 6.7B 到 15B，涵盖了不同领域：通用模型有 Deepseek-llm-7b-chat、Meta-Llama-3.1-8B-Instruct、Qwen2.5-7B-Instruct、Qwen2.5-14B-Instruct；代码模型有 Deepseek-coder-7b-instruct-v1.5、CodeQwen1.5-7B-Chat、Qwen2.5-Coder-7B-Instruct；SQL 专用模型有 CodeS-7B 和 CodeS-15B 。这些模型涵盖了当前主流的中小规模开源 LLM，让研究人员的结论更具普适性。

所有实验都在配备 8 张 NVIDIA A800 GPU（80GB 内存）的服务器上进行，使用 CUDA 12.1.1。训练框架采用 Llama Factory 并启用了 FlashAttention 2.0 和 DeepSpeed ZeRO-3 加速训练、减少显存占用。对于超过 10B 参数的模型，在 DPO 阶段研究人员启用了 CPU 卸载以容纳优化器状态。

训练超参数方面，研究人员采用全参数微调，使用 bf16 混合精度。优化器是 AdamW，$\beta_1 = 0.9$，$\beta_2 = 0.99$。学习率调度采用余弦衰减，前 5% 的步骤进行线性预热。批次大小统一设为 64，上下文窗口为 4096 个词元。学习率在 SFT 阶段，对于小于 10B 的模型设为 1×10^{-5}，大于 10B 的设为 7×10^{-6}；在 DPO 阶段则分别设为 1×10^{-6} 和 7×10^{-7}。DPO 的超参数 β 默认设为 0.1（后面研究人员还会专门分析它的敏感性）。训练轮数上，SFT 阶段训练 4 个 epoch，研究人员根据开发集上的 Maj@16 指标选出最佳检查点作为 DPO 的参考模型；DPO 阶段继续训练 8 个 epoch。

数据构建也有一些讲究。输入提示的平均长度为 965 个词元，里面包含了问题、数据库模式信息（表名、列名、主外键）以及通过模式链接和值匹配检索出的相关表和列值。研究人员沿用了 CodeS 的方法：先用一个模式项分类器（基于 XLMRoBERTa-XL，3.5B）识别最相关的表和列，然后通过“粗到细”的匹配方法从数据库中检索与问题相关的值，最后组合成提示。输出标签方面，原始数据集的输出是标准 SQL 查询，平均长度 44 个词元；而 CoT 增强后，输出的 CoT 解决方案平均长度 404 个词元（词元数由 Qwen2.5-7B-Instruct 的分词器测量）。CoT 合成研究人员使用 GPT-4o-mini-2024-07-18，每个样本生成 16 个 CoT 解决方案，提示模板见附录 B。偏好数据构建则用 SFT 模型（π_{SFT}）为每个训练样本生成 16 个响应，根据执行结果标记正确/错误，然后随机配对形成偏好对。对于 Bird，最终偏好数据集大小约为 1.5k 到 2.5k 对，具体数量取决于不同模型。

2.4 实验结果

研究人员首先在 Bird 基准上展示主要发现，然后深入错误分析，最后在 Spider 及其变体上验证结论。

2.4.1 主要结果：Bird 开发集

研究人员对比了两种训练方式：一种是 **Vanilla**，即直接在原始 Bird 数据上进行 SFT 和 DPO；另一种是 **Syn CoT**，即在 CoT 增强数据上进行 SFT 和 DPO。对于每种方式，研究

人员报告了 SFT 和 DPO 阶段在三种推理策略（Greedy, Pass@1, Maj@16）下的执行准确率（EX），以及“Syn CoT + DPO”相对于“Vanilla + SFT”的增益（ΔEX）。

第一个观察是：原始模型在 DPO 阶段很难获得性能提升。大多数模型在 DPO 后的准确率变化微乎其微，甚至出现下降。这说明 DPO 并没能让模型变得更可靠。第二个观察则让人眼前一亮：使用合成 CoT 的模型在 DPO 阶段获得了稳定且显著的增益。再看 Syn CoT 部分，所有模型在 DPO 后都有了明显进步，说明模型不仅更准，也更可靠了。第三个观察是：合成 CoT 加 DPO 展现出了更高的性能上限。所有 Syn CoT 模型的最终性能都远超对应的 Vanilla + SFT 基线。

2.4.2　错误分析：CoT 如何帮助 DPO 纠正错误？

为了更深入地理解 DPO 对模型生成的影响，研究人员以 Qwen2.5-7B-Instruct 为例，用贪心解码对它在开发集上的预测错误进行了细致的分类。研究人员把错误分成了 6 大类 17 小类，每类的详细描述见附录 I。表 3 展示了每个错误类型在 DPO 后的修正率（Fix %），也就是 DPO 后正确而 SFT 时错误的比例。

研究人员发现，DPO 特别擅长纠正那些因忽略细节而引起的错误。修正率超过 25% 的错误类型在表中用粗体标出，比如 NULL/DISTINCT（用 CoT 修正了 40.0%）和列顺序（用 CoT 修正了 42.9%）。这类错误往往是因为模型没有仔细处理数据中的空值、重复值，或者没有按问题要求返回正确的列顺序。而 CoT 的逐步推理迫使模型在生成 SQL 之前先考虑这些细节，从而帮助 DPO 更好地识别和纠正。

CoT 还能提供逻辑指导，增强 DPO 的错误纠正能力。对于涉及多表连接的 JOIN 错误，Syn CoT DPO 的修正率达到了 32.1%，比 Vanilla DPO 高出了 16.5 个百分点。这类错误通常需要理解表之间的关系，CoT 的逐步逻辑有助于理清连接路径。相比之下，对于幻觉（Hallucination）和日期处理（Date）这类直观错误，CoT 的增益就小一些（分别 +3.5% 和 +7.3%），因为原始 DPO 已经能处理得不错了。

2.4.3　Spider 及其变体的结果

总体的趋势和 Bird 一致：Syn CoT + DPO 在 Spider 开发集以及各种鲁棒性基准上都显著优于 Vanilla + DPO，证明了这个方法有很好的泛化能力。不过研究人员也注意到，Spider 上 DPO 的增益略小于 Bird，这可能是因为 Spider 的 SQL 查询相对简单，而且数据库行数少导致执行反馈不够精确（容易出现假阳性）。

接下来，研究人员以 Qwen2.5-7B-Instruct 为对象，在 Bird 基准上进行了一系列分析，试图弄清楚 CoT 对 DPO 的关键作用。研究人员从三个维度来深入探讨：判别能力、训练稳定性、可扩展性。

2.4.4　更好的判别能力

DPO 有个重要特性，就是它隐式学习了一个奖励模型，可以用来评估生成回答的好坏。一个优秀的 DPO 模型应该能准确地区分正确和错误的回答。为此，研究人员构建了一个评估偏好数据集，其中每个样本包含一个正确和一个错误的 CoT 解决方案（对于 Syn CoT 模型）或者对应的 SQL（对于 Vanilla 模型）。然后研究人员计算 DPO 模型根据隐式奖励选择正确响应的准确率。Syn CoT 模型的准确率稳定在较高水平（接近 90%），而 Vanilla 模型的准确率波动较大，而且整体偏低。这表明 CoT 推理让 DPO 模型能够更可靠地判断回答的质量。

2.4.5　更好的训练稳定性

训练稳定性是衡量 DPO 效果的一个重要指标。研究人员考察了两个现象：奖励破解和超参数敏感性。

先看奖励破解。奖励破解（Reward Hacking）是指模型在训练过程中学会通过生成一些看似合理但实际错误的输出来获取高奖励，从而欺骗奖励模型。在 DPO 中，研究人员可以通过观察模型在开发集上的真实性能与模型自我评估的隐式奖励之间的差异来检测奖励破解。

对于原始模型，随着训练进行，模型在开发集上的准确率先升后降（在第 2 个 epoch 达到峰值后就开始下降），但隐式奖励却一路走高。这说明模型觉得自己的输出越来越好，可实际上性能在下滑，这正是奖励破解的典型表现。而对于 Syn CoT 模型，准确率先升后趋于饱和，隐式奖励也随之先升后平稳，和真实性能高度一致。这表明 CoT 有效防止了模型走捷径，让它学习到的奖励与真实性能对齐了。

研究人员进一步分析了模型输出的统计特性。原始模型在 DPO 后，平均 SQL 字符长度从 1732 猛增到 1917（增长了 26.6%），同时不可执行 SQL 的比例从 2.0% 飙升到 19.6%（增长了 17.6%）。这说明模型试图通过生成长而复杂的 SQL 来获取高奖励，但这些 SQL 往往根本没法执行。相反，Syn CoT 模型的 SQL 长度只微增了 2%，不可执行率反而从 7.6% 降到了 5.1%，说明它的输出更可靠。

2.4.6　更好的可扩展性

研究人员还评估了 Syn CoT DPO 在偏好数据规模（采样预算）和推理时计算（采样数量）方面的可扩展性。图 5a 显示，对于原始模型，增加偏好数据收集时的采样预算（也就是从每个训练样本生成更多响应来构建偏好对）并不会提升 DPO 性能；而对于 Syn CoT 模型，性能却随着预算增加而持续提升。图 5b 展示了推理时采用 Maj@K 策略时，增加采样数 K 对性能的影响。Syn CoT DPO 在 K 较小时（比如 K=2）就能达到较高的准确率，而原始 DPO 需要 K=16 才能接近这个水平。这表明 CoT 让模型能够在有限的推理预算下产生更可靠的输出。

2.5　前景：集成到现有 Text-to-SQL 框架中

研究人员训练的 DPO 模型可以作为一个更强的基座模型，集成到现有的多智能体 Text-to-SQL 框架中，进一步提升整体性能。

研究人员选取 Qwen2.5-7B-Instruct 作为基础模型，把它集成到两个有代表性的框架里。一个是 **DTS-SQL (Pourreza and Rafiei, 2024b)**，它把 Text-to-SQL 分解成模式链接和 SQL 生成两个阶段，研究人员替换了它 SQL 生成阶段的模型。另一个是 **C3-SQL (Dong et al., 2023)**，它包含了清晰提示、模型偏差校准和一致性输出三个模块，研究人员替换了它的核心生成模型。

用 Syn CoT + DPO 模型替换原框架的基座模型后，两个框架在 Spider 开发集上的准确率都有了提升，这证明了研究人员训练的模型确实有实用价值。

2.6　结论

回顾这项工作的整个探索历程，研究人员首次证明了通过合成思维链（CoT）解决方案，可以在 Text-to-SQL 任务上实现直接偏好优化（DPO）的一致且有效的性能改进。通过全面的实验和深入的分析，研究人员揭示了一个关键发现：CoT 推理对于释放 DPO 在 Text-to-SQL 中的潜力至关重要，因为它能缓解奖励破解、增强模型判别能力、提高训练可扩展性。这些见解不仅解释了为什么先前将 DPO 直接应用于 Text-to-SQL 会失败，也为未来构建更强大的开源 Text-to-SQL 模型指明了方向。研究人员相信，CoT 与偏好优化的结合将为复杂推理任务带来更深远的影响。

2.7　局限性

尽管取得了显著的成果，研究人员也很清楚这项研究还存在一些局限性。首先是执行时间问题。Bird 数据集中的数据库通常很大，执行 SQL 查询非常耗时，尤其是在偏好数据构建阶段需要执行大量采样查询。研究人员虽然通过多进程并行执行加快了速度，但多进程间的资源争用可能导致部分查询超时，使一些本应正确的 SQL 被误判为错误，引入假阴性，这可能会影响 DPO 的训练质量。其次是反馈不准确性的问题。Spider 数据集则相反，因为数据库行数少，执行结果匹配容易产生假阳性（比如两个不同的 SQL 返回相同的结果），导致偏好信号不可靠。未来可以考虑结合测试套件（如 TS）来构建更准确的反馈信号。另外，CoT 合成质量也是一个值得探讨的方向。研究人员主要用 GPT-4o-mini 作为合成器，虽然成本低而且效果显著，但更高质量的 CoT（比如人工标注或者用更强模型合成）可能会带来进一步提升。同时，研究人员也没有深入探讨 CoT 的风格（比如语言、详细程度）对 DPO 的影响，这可以作为未来研究的一个方向。最后是模型规模的问题。本研究主要聚焦于 6.7B 到 15B 参数的中小规模模型，对于更大模型（比如 30B 以上）是否同样适用还有待验证。不过研究人员相信，方法论本身应该是可以扩展的。